

Experiment No: 7

Title

Write a Python Program to Implement Breadth First Search (BFS)

Aim

To develop a Python program to implement the **Breadth First Search (BFS)** algorithm for finding the shortest path between a source node and a destination node in a graph.

Objective

To implement the Breadth First Search (BFS) algorithm using a queue to traverse the graph level by level and determine the shortest path between two given nodes.

Algorithm

Step 1: Start the program.

Step 2: Define the graph using an adjacency list.

Step 3: Read the source node and destination node.

Step 4: Initialize a queue with the source node and its path.

Step 5: Create an empty set to store visited nodes.

Step 6: Remove the front node from the queue.

Step 7: If the current node is the destination node, display the shortest path and stop.

Step 8: Otherwise, visit all unvisited neighbouring nodes and insert them into the queue.

Step 9: Repeat Steps 6–8 until the queue becomes empty.

Step 10: If no path exists, display an appropriate message.

Step 11: Stop the program.

Program

```
from collections import deque
```

```
# Breadth First Search Function
```

```
def bfs(graph, start, goal):
```

```
queue = deque([(start, [start])])
```

```
visited = set()
```

```
while queue:
```

```
    node, path = queue.popleft()
```

```
    if node == goal:
```

```
        return path
```

```
    if node in visited:
```

```
        continue
```

```
    visited.add(node)
```

```
    for neighbour in graph[node]:
```

```
        if neighbour not in visited:
```

```
            queue.append((neighbour, path + [neighbour]))
```

```
return None
```

```
# Main Program
```

```
graph = {
```

```
    0: [1, 2],
```

```
    1: [0, 2, 3],
```

```
    2: [0, 1, 3],
```

```
    3: [1, 2, 4],
```

```

    4: [3]
}

print("Graph Representation")
print("-----")

for node in graph:
    print(node, "->", graph[node])

print()

start = int(input("Enter Source Node    : "))
goal = int(input("Enter Destination Node : "))

path = bfs(graph, start, goal)

print()

if path:

    print("Breadth First Search Successful")
    print("-----")
    print("Source Node    :", start)
    print("Destination Node :", goal)
    print("Shortest Path    :", " -> ".join(map(str, path)))
    print("Number of Nodes Visited :", len(path))

else:
    print("No Path Found")

```

Sample Input

Enter Source Node : 0

Enter Destination Node : 4

Sample Output

```
----- BFS -----
Graph Representation
-----
0 -> [1, 2]
1 -> [0, 2, 3]
2 -> [0, 1, 3]
3 -> [1, 2, 4]
4 -> [3]

Enter Source Node      : 0
Enter Destination Node : 4

Breadth First Search Successful
-----
Source Node      : 0
Destination Node : 4
Shortest Path    : 0 -> 1 -> 3 -> 4
Number of Nodes Visited : 4
|
```

Result

Thus, the Python program to implement the **Breadth First Search (BFS)** algorithm was executed successfully, and the shortest path between the source node and destination node was obtained correctly.